

Pgeon: Generating Tableau-Based Provers from Declarative Specifications of Logical Calculi

Romain Sidhoum, Simon Robillard, David Delahaye

▶ **To cite this version:**

Romain Sidhoum, Simon Robillard, David Delahaye. Pgeon: Generating Tableau-Based Provers from Declarative Specifications of Logical Calculi. IJCAR 2026 - 13th International Joint Conference on Automated Reasoning, Jul 2026, Lisbonne, Portugal. pp.98-105, ⟨10.1007/978-3-032-32589-1_6⟩. ⟨hal-05802961⟩

HAL Id: hal-05802961

https://hal.science/hal-05802961v1

Submitted on 16 Oct 2026

HAL is a multi-disciplinary open access archive for the deposit and dissemination of scientific research documents, whether they are published or not. The documents may come from teaching and research institutions in France or abroad, or from public or private research centers.

L'archive ouverte pluridisciplinaire **HAL**, est destinée au dépôt et à la diffusion de documents scientifiques de niveau recherche, publiés ou non, émanant des établissements d'enseignement et de recherche français ou étrangers, des laboratoires publics ou privés.



Copyright - All rights reserved

Pgeon: Generating Tableau-Based Provers from Declarative Specifications of Logical Calculi

Romain Sidhoum, Simon Robillard, and David Delahaye

LIRMM, Univ Montpellier, CNRS, Montpellier, France
Firstname.Lastname@lirmm.fr

Abstract. This paper introduces Pgeon, a meta-prover framework that generates tableau-based automated theorem provers from declarative specifications. In Pgeon, the syntax of a given calculus, its tableau inference rules, and the proof-search strategy are described in a small domain-specific language that closely follows textbook presentations. From such a specification, Pgeon instantiates a fully functional prover, handling tableau construction, rule instantiation, branching, and backtracking in a logic-agnostic manner. Proof-search is driven by a strategy engine that gives users explicit control over exploration order while keeping logical content separate from operational concerns. Pgeon supports first-order reasoning through binders, capture-avoiding substitution, and extensible term generators required for Skolemization and free variable introduction. We describe the design of the specification language, the execution model of the tool, and the strategy mechanism, and illustrate the approach on case studies covering classical first-order tableau calculus and intuitionistic propositional calculus.

Keywords: Tableau Calculi · Automated Prover Generation · Logical Framework.

1 Introduction

Many Automated Theorem Provers (ATPs) have been implemented, covering a broad range of techniques (resolution, tableaux, superposition, DPLL, etc.), as well as a variety of target logics. These systems are usually highly specialized and optimized with implementations that often intertwine proof rules and search control, resulting in code that is difficult to reuse. The development of a new prover is a challenge that requires a deep understanding of the implemented calculus, as well as practical programming skills. As a result theoretical developments in deductive systems are not always implemented.

Pgeon (short for Prover GEneratiON) is a tool designed to help quickly develop an ATP. It offers a DSL that allows the user to specify the syntax of a logic, proof production rules, and the proof-search strategy that controls those rules. From that description, Pgeon generates an executable prover able to handle term management, branch expansion, and backtracking. Pgeon is equipped with features to handle quantified logics, including the ability to specify binders

in the syntax, rules with unification and substitution, and proof-search strategies with fine-tuned control over the application of rule instances and backtracking. Although logic-agnostic, Pgeon is primarly intended to generate tableau-based provers. More generally, the rules describe transformation steps on proof trees and their nodes. The description language of Pgeon is meant to resemble textbook notation and provide a natural framework for researchers to prototype or experiment with new deduction techniques.

This paper presents the design and current implementation of Pgeon. We describe the specification language and the proof-search engine. Along the way, we report on case studies ranging from classical first-order tableau calculus to intuitionistic propositional calculus, outlining what makes Pgeon[1] a versatile framework for automated proving research.

Related Work For background on tableau methods we refer to the Handbook of Tableau Methods [5]. This work is inspired by previous tableau prover generators, in particular LoTREC [4], Tableau workbench [1] and MetTeL [10]. Unlike Pgeon, these systems are limited to propositional logics. Prolog has been used to provide elegant and concise implementations of provers for first-order logic, based on semantic tableaux [3], as well as connection tableaux [8], although in both cases the evaluation strategy is largely constrained by the semantics of Prolog. A different approach is taken by the first-order tableau prover KeY, namely allowing users to extend the calculi with new rules through a dedicated description language [2]. Sequoia [9] is another system with a dedicated description language for (propositional) logical calculi, intended for meta-logical analysis rather than execution. Our strategy language is also related to tactic languages in interactive theorem proves, such as Ltac, the tactic language of Rocq, which provides combinators and constructs for composing proof-search procedures [6].

2 Specification Language and Usage

Pgeon generates tableau provers from a declarative specification consisting of a file that defines the syntax, inference rules and a proof-search strategy of a given logic. Problems are defined in separate files that extend the syntax with the symbols occurring in the problem, and provide the formulas from which the proof is started. Listing 1.1 shows the specification of a tableau calculus for classical first-order logic, from which Pgeon generates an executable prover. We refer to this specification as a running example throughout this section.

Syntax. This file starts with type declarations, corresponding to syntactic categories (here: `formula` and `term`). Types are used to enforce well-formedness in symbol declarations and rules. One type must be prefixed with the keyword `main`, thereby designating it as the type of elements in the proof tree nodes. The syntactic constructors of the object language are declared with the keyword

[1] Pgeon is an open source tool, available at https://gite.lirmm.fr/rsidhoum/pgeon

```
main type formula
type term

function not  : formula -> formula
function and  : formula formula -> formula
function or   : formula formula -> formula
function impl : formula formula -> formula

binder forall : term. formula
binder exists : term. formula

rule rAxiom   : P ; not(P) ==X
rule rAnd     : and(P, Q)       ==> P ; Q
rule rOr      : or(P, Q)        ==> P | Q
rule rImpl    : impl(P, Q)      ==> not(P) | Q
rule rNotNot  : not(not(P))     ==> P
rule rNotAnd  : not(and(P, Q))  ==> not(P) | not(Q)
rule rNotOr   : not(or(P, Q))   ==> not(P) ; not(Q)
rule rNotImpl : not(impl(P, Q)) ==> P ; not(Q)
tree rule rClosure : (P0 ; not(P1) ; ...B) | ...T --> ...T'
where {
  T' = ((P0; not(P1); ...B) | ...T)[mgu(P0, P1)]
}
rule rForall : forall x. P --> Q
where {
  Q = P[x <- @fresh()]
}
rule rNotForall : not(forall x. P) ==> not(Q)
where {
  Q = P[x <- @skolem()]
}
rule rNotExists : not(exists x. P) --> not(Q)
where {
  Q = P[x <- @fresh()]
}
rule rExists : exists x. P ==> Q
where {
  Q = P[x <- @skolem()]
}

strategy closure : rAxiom || (rClosure; rAxiom)
strategy alpha : rAnd || rNotNot || rNotOr || rNotImpl
strategy beta : rOr || rImpl || rNotAnd
strategy delta : rExists || rNotForall
strategy gamma : rForall!? ; rNotExists!?
main strategy prove :
    (closure || alpha || delta || beta || gamma)*
```

Listing 1.1. Classical First-Order Tableau Calculus Specification in Pgeon

`function`, with their type signature (`not`, `and`, `or`, `impl` map formulas to formulas). The syntax may also include `binder`s (here `forall`, `exists`, that build `formula`s with a bound `term`), allowing the definition of quantified logics.

Rules. Rules describe admissible transformations on tableau branches. Closure rules (`==X`) mark a branch as closed, invertible rules (`==>`) remove matched nodes in a branch to extend it with new nodes, and non-invertible rules (`-->`) extend the branch with new nodes, without removing the matched nodes. Each rule has a name, a left-hand side pattern, and (except for closure rules) a right-hand side that specifies the generated nodes. Patterns use meta-variables (e.g.,

`P`, `Q`) that match nodes occurring on a branch. Some rules may require the presence of multiple nodes in the branch (separated by the operator `;` on the left-hand side), e.g., the rule `rAxiom`. For such patterns, the order of nodes is irrelevant, i.e., a branch is treated as a multiset of nodes. On the right-hand side, the semicolon (`;`) adds formulas to the same branch, and the vertical bar (`|`) denotes branching. Here, the rule `rAnd` decomposes a conjunction on the same branch, whereas `rOr` creates a branching point. For inferences that require global tableau information, `tree rule`s match and modify the tableau, rather than a single branch. The ellipses "`...`" denote patterns that match an arbitrary number of formulas or branches. Here, the `rClosure` rule implements unification-based closure: it searches for complementary formulas up to unification and applies the most general unifier to the whole tableau. Rules can have side-conditions and computational definitions in a block `where`. The notation `t[x <- e]` is used to perform capture-avoiding substitution, and calls to external Ocaml functions are indicated with `@`. We provide utility functions for first-order logic, such as `fresh` (that generates a fresh free variable) and `skolem` (that generates a Skolem term parameterized by the free variables in the current branch). Additional external functions can be defined by the user to perform complex operations outside the scope of Pgeon's description language.

Strategies. Strategies define how rules are applied during proof-search. One strategy must be prefixed with the keyword `main`, designating it as the strategy executed by the evaluation engine. A rule call is interpreted as a function from proof states to a sequence of successor states. The sequence may be empty, when no instance of the rule is applicable, or may contain several states, when the rule has several possible matches. Strategy operators combine such state transformers:

- `S1 || S2` is left-biased choice: `S1` is tried first, and `S2` is tried only if `S1` produces no successor.
- `S1 ; S2` is sequential composition: `S2` is applied to each successor produced by `S1`.
- `S?` is optional execution: it behaves as `S` when `S` succeeds, and as the identity strategy otherwise.
- `S*` denotes repeated application of the strategy `S`. It explores states reachable by zero or more applications of `S`
- `R!` applies every currently available instance of the rule `R` once. Unlike `*`, `!` takes a rule as argument, not an arbitrary strategy. It does not recursively apply to the states it produces.

The strategy definitions in Listing 1.1 group non-branching rules, branching rules, quantifier rules, and closure rules, and compose them in the main strategy.

3 Engine Architecture and Execution Model

Pgeon builds proofs by interpreting user-defined strategies over an evolving tableau. A tableau is represented as a tree structure whose nodes are individual formulas. Each branch corresponds to a multiset of formulas. A branch is

considered closed if a closure rule has been applied to it. The tableau is closed when each of its branch is closed. Proof search is implemented by interpreting strategies over proof states. A proof state consists of a tableau together with a strategy that remains to be executed. Strategy evaluation may fail, produce one successor state, or produce several alternatives. These alternatives are stored as continuations and explored by backtracking. Thus, non-determinism arises both from explicit strategy operators and from the possible instantiations of a rule.

Rule Instantiation. Rule application proceeds by searching for matches of the rule's left-hand side pattern in the current tableau. For ordinary rules, matching is performed on individual branches, viewed as multisets of formulas. For tree rules, matching is performed on the whole tableau, allowing transformations that depend on several branches or on the global shape of the proof tree. Once a match is found, the rule body and its side-conditions are evaluated under the computed substitution. The tableau is then updated according to the kind of rule: closure rules mark branches as closed, invertible rules replace matched formulas by their conclusions, non-invertible rules add conclusions while preserving the matched formulas, and tree rules may rewrite the tableau globally. Each successful instantiation yields a new proof state. When several matches are possible, the engine schedules the corresponding states as alternatives, thereby integrating rule-level non-determinism into the global search.

Backtracking. Backtracking arises from explicit choice points and alternative rule matches. When a rule has several admissible instances, each instance produces a successor state, and the resulting stream represents the corresponding choice points. The default exploration order is depth-first, following the order in which successor states are produced. Consequently, termination and fairness are not guaranteed for arbitratry strategies. This is expected for first-order proof search, where rules such as quantifier instantiation may generate infinitely many successors. The purpose of the strategy language is therefore not to guarantee termination automatically, but to make the search policy explicit.

For example, iterative deepening can be expressed by combining repetition with sequential composition. If `gamma` denotes quantifier instantiation and `sat` a saturation phase for the remaining rules, then `(gamma* ; sat)*` schedules saturation after zero, one, two, and progressively more applications of over `gamma`. Provided that the results of `gamma` are enumerated fairly, this prevents an infinite sequence of quantifier instantiations from starving the other rules.

4 Description of Another Use-Case

Section 2 already provides a complete calculus example, namely tableaux for classical first-order logic with Skolemization and free variables, in a destructive version, i.e., the use of the closure rule generates a substitution that is applied to the entire tree. In this section, we will look at a less straightforward example, namely intuitionistic propositional logic. While this logic remains decidable,

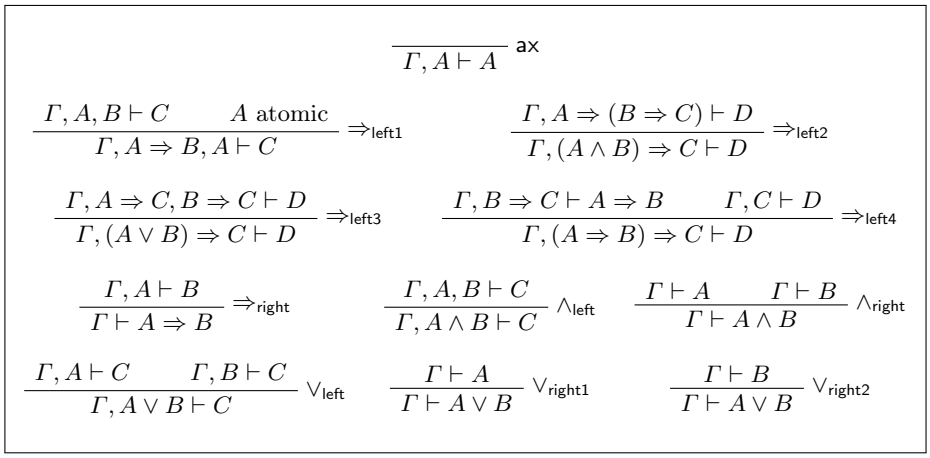


Fig. 1. LJT Rules for Intuitionistic Propositional Logic

expressing the corresponding sequent calculus in an automation-friendly system presents some difficulties. The first difficulty lies in the intuitionistic semantics of the sequent, where there is a single formula on the right. It is not possible to put this formula on the left (with a negation) because that would amount to applying the excluded middle. This means that the formulas need to be tagged (left or right) in order to emulate a sequent, which Pgeon allows us to do.

The other difficulty lies in the use of contraction (on the left). While contraction (on the left or right) can be removed in classical propositional logic, this is not the case in intuitionistic logic. Left contraction can be removed everywhere except for the left implication rule. Since the goal is to automate the search for proof, this rule is problematic because it is a source of non-termination. Roy Dyckhoff proposed a system, called LJT [7], which replaces this rule with four other rules $\Rightarrow_{\mathsf{left}i}$ with $i = 1, \ldots, 4$, allowing contraction to be eliminated completely. The entire LJT system is presented in Figure 1. Note that to simplify the presentation, we adhere to Dyckhoff's original system by not considering top, bottom, negation, and equivalence. It should also be noted that all rules are reversible except for the rules $\vee_{\mathsf{right1}}$, $\vee_{\mathsf{right2}}$, and $\Rightarrow_{\mathsf{left4}}$.

The specification in Pgeon of the LJT rules is given in Listing 1.2. Note the existence of the two tags left and right, which allow us to emulate the intuitionistic sequent as mentioned above. Furthermore, since contraction is no longer necessary, we use "`==>`" for all rules (except the axiomatic rule), which allows us to produce new sequents without the matched formulas. In addition to the problem of contraction, this type of rule allows for the management of potential weakening, as in rule $\Rightarrow_{\mathsf{left4}}$, where D in the conclusion of the rule is removed from the left branch of the premises. This allows the rules of the LJT system to be written almost directly without transforming the system. Non-reversible rules are handled by Pgeon's backtracking mechanism, described earlier in Section 3.

```
type prop
main type formula

function and : prop prop -> prop
function or  : prop prop -> prop
function imp : prop prop -> prop
function left  : prop -> formula
function right : prop -> formula

rule ax : left(P0) ; right(P0) ==X
rule and_right : right(and(P0, P1)) ==> right(P0) | right(P1)
rule and_left : left(and(P0, P1)) ==> left(P0) ; left(P1)
rule or_right1 : right(or(P0, P1)) ==> right(P0)
rule or_right2 : right(or(P0, P1)) ==> right(P1)
rule or_left : left(or(P0, P1)) ==> left(P0) | left(P1)
rule imp_right : right(imp(P0, P1)) ==> left(P0) ; right(P1)
rule imp_left1 : left(imp(P0, P1)) ; left(P0) ==>
  left(P0) ; left(P1)
rule imp_left2 : left(imp(and(P0, P1), P2)) ==>
  left(imp(P0, imp (P1, P2)))
rule imp_left3 : left(imp(or(P0, P1), P2)) ==>
  left(imp(P0, P2)) ; left(imp(P1, P2))
rule imp_left4 : left(imp(imp(P0, P1), P2)) ; right(P3) ==>
  left(imp(P1, P2)) ; right(imp(P0, P1))
| left(P2) ; right(P3)

strategy reduce_once : ax || and_right || and_left
  || or_right1 || or_right2 || or_left || imp_right
  || imp_left1 || imp_left2 || imp_left3 || imp_left4
strategy reduce_all : reduce_once*
strategy reduce_fixed : (reduce_once ; reduce_fixed)?
main strategy prove : reduce_all ; reduce_fixed?
```

Listing 1.2. LJT Specification in Pgeon

5 Conclusion and Future Work

This paper introduces Pgeon, a framework designed to generate tableau-based provers from declarative logical specifications. By separating logical content from implementation, users can describe syntax, inference rules, and proof strategies in a lightweight DSL that mirrors textbook presentations. Specifications are compiled into executable provers via a logic-agnostic engine.

Case studies covering classical first-order tableau calculus and intuitionistic propositional calculus show that Pgeon enables rapid prototyping and experimentation, with strategies providing compositional control over proof-search. While flexible, Pgeon prioritizes clarity over efficiency: termination is not ensured and empirical evaluation remains to be carried out.

Future work includes improving efficiency through optimized indexing and term-sharing structures, supporting associative and associative-commutative operators, adding proof trace generation, and exploring additional logics such as modal logic. We would also like to introduce the concept of sequent in a primitive way (we have seen that this concept can already be encoded in the LJT example), which would increase the versatility of the tool by offering the possibility of directly implementing sequent calculi (in particular systems that do not have an associated tableau calculus that is easy to implement).

Disclosure of Interest The authors have no competing interests to declare.

References

1. Abate, P., Goré, R.: The tableau workbench. Electronic Notes in Theoretical Computer Science **231**, 55–67 (2009)
2. Beckert, B., Giese, M., Habermalz, E., Hähnle, R., Roth, A., Rummer, P., Schlager, S.: Taclets: A new paradigm for constructing interactive theorem provers. RACSAM **98**(1), 17–53 (2004)
3. Beckert, B., Posegga, J.: leanTAP: Lean Tableau-Based Deduction. Journal of Automated Reasoning **15**(3), 339–358 (1995)
4. del Cerro, L.F., Fauthoux, D., Gasquet, O., Herzig, A., Longin, D., Massacci, F.: Lotrec: the generic tableau prover for modal and description logics. In: International joint conference on automated reasoning. pp. 453–458. Springer (2001)
5. D'Agostino, M., Gabbay, D.M., Hähnle, R., Posegga, J.: Handbook of tableau methods. Springer Science & Business Media (2013)
6. Delahaye, D.: A tactic language for the system coq. In: International Conference on Logic for Programming Artificial Intelligence and Reasoning. pp. 85–95. Springer (2000)
7. Dyckhoff, R.: Contraction-Free Sequent Calculi for Intuitionistic Logic. Journal of Symbolic Logic **57**, 795 – 807 (Sep 1992)
8. Otten, J., Bibel, W.: leanCoP: Lean Connection-Based Theorem Proving. Journal of Symbolic Computation **36**(1-2), 139–161 (2003)
9. Reis, G., Naeem, Z., Hashim, M.: Sequoia: A playground for logicians: (system description). In: International Joint Conference on Automated Reasoning. pp. 480–488. Springer (2020)
10. Tishkovsky, D., Schmidt, R.A., Khodadadi, M.: The tableau prover generator mettel2. In: Logics in Artificial Intelligence. pp. 492–495. Springer (2012)